ASSIGNMENT- 10.5

Name: K.Archana

HT.No: 2303A51329

Batch: 20

Task 1: Variable Naming Issues .Improve unclear variable and function names using AI.

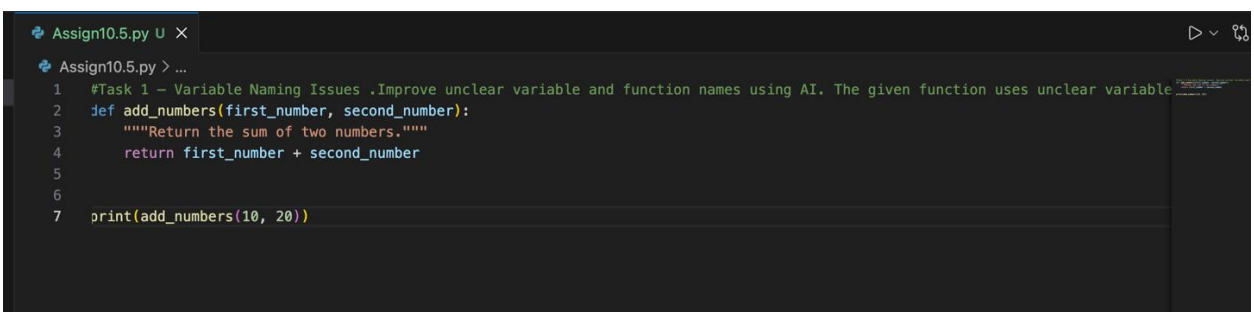
The given function uses unclear variable names (`f`, `a`, `b`), which reduce readability and maintainability.

```
def f(a, b):
return a + b
print(f(10, 20))
```

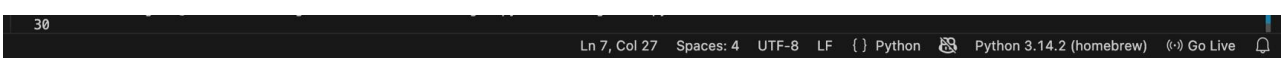
Prompt: #Refactor the function to use meaningful function and variable

names following PEP 8 standards.

Code:



Result:



Observation:
The refactored code improves readability by using descriptive names. The function name clearly indicates its purpose. Adding a docstring improves documentation. The updated version follows PEP 8 naming conventions and enhances maintainability

Task 2: Missing Error Handling. Add proper exception handling.

The original code crashes when dividing by zero because no error handling is implemented.

def divide(a, b):
return a / b
print(divide(10, 0))

Prompt: #Add proper error handling and clear

error messages to the divide function.

Code:

```
#Task 2: Missing Error Handling. Add proper exception handling. The original code crashes when dividing by zero because no error
def divide_numbers(numerator, denominator):
    """Divide two numbers with error handling."""
    try:
        return numerator / denominator
    except ZeroDivisionError:
        return "Error: Division by zero is not allowed."
    except TypeError:
        return "Error: Invalid input type."


print(divide_numbers(10, 0))
```

Result:

```
30
Error: Division by zero is not allowed.
```

Observation:

The AI-enhanced version prevents program crashes by adding exception handling. It provides meaningful error messages instead of terminating execution. This improves robustness and user experience.

Task 3: Student Marks Processing System. Refactor code to follow PEP 8 standards and improve readability.

```
marks=[78,85,90,66,88]
t=0
for i in marks:
t=t+i
a=t/len(marks)
if a>=90:
print("A")
elif a>=75:
print("B")
elif a>=60:
print("C")
else:
print("F")
```

The original program lacks structure, meaningful variable names, and input validation.

Prompt: # Refactor the student marks processing system to follow PEP 8, add meaningful variable names, functions, comments, and basic validation.

Code:

```
#Task 3:  Student Marks Processing System. Refactor code to follow PEP 8 standards and improve readability. The original program
def calculate_grade(marks):
    """
    Calculate total, average, and grade based on marks list.
    """
    if not marks:
        return "No marks provided."

    total_marks = sum(marks)
    average_marks = total_marks / len(marks)

    if average_marks >= 90:
        grade = "A"
    elif average_marks >= 75:
        grade = "B"
    elif average_marks >= 60:
        grade = "C"
    else:
        grade = "F"

    return total_marks, average_marks, grade


marks_list = [78, 85, 90, 66, 88]
result = calculate_grade(marks_list)

if isinstance(result, tuple):
    total, average, grade = result
    print("Total:", total)
    print("Average:", round(average, 2))
    print("Grade:", grade)
```

Result:

```
30
Error: Division by zero is not allowed.
Total: 407
Average: 81.4
Grade: B
```

Observation:

The AI refactored the code into a structured function, improving modularity and reusability. Meaningful variable names increase clarity. Input validation ensures safety. The code now follows PEP 8 guidelines and improves maintainability.

Task 4: Adding Docstrings and Inline Comments. Add documentation and comments.

The factorial function lacks documentation and explanatory comments.

def factorial(n):
result = 1
for i in range(1,n+1):
result *= i
return result

Prompt: # Add a proper docstring and inline comments to the factorial function.

Code:

```
#Task 4: Adding Docstrings and Inline Comments. Add documentation and comments. The factorial function lacks documentation and c


def factorial(number):
    """
    Calculate the factorial of a non-negative integer.

    Parameters:
    number (int): A non-negative integer

    Returns:
    int: Factorial of the number
    """
    if number < 0:
        return "Factorial is not defined for negative numbers."

    result = 1
    for i in range(1, number + 1):
        result *= i  # Multiply result by current number

    return result


print(factorial(5))
```

Result:

```
30
Error: Division by zero is not allowed.
Total: 407
Average: 81.4
Grade: B
120
```

Observation

The addition of docstrings and inline comments improves readability and documentation quality. The function now explains its purpose, parameters, and return values clearly.

Task 5: Enhanced Password Validation System. Improve password validation with stronger security rules.

The original code checks only minimum length, which is insufficient for security.

pwd = input("Enter password: ")
if len(pwd) >= 8:
print("Strong")
else:
print("Weak")

Prompt: # Enhance the password validation program to include:

- Minimum length

- At least one uppercase letter

- At least one lowercase letter

- At least one digit

- At least one special character

Follow PEP 8 and add comments and docstring.

Code:

```python
import re
def validate_password(password):
    """
    Validate password strength based on multiple security rules.
    Returns True if strong, otherwise False.
    """

    if len(password) < 8:
        return False

    if not re.search(r"[A-Z]", password):
        return False

    if not re.search(r"[a-z]", password):
        return False

    if not re.search(r"[0-9]", password):
        return False

    if not re.search(r"[!@#$%^&*(),.?\":{}|<>]", password):
        return False

    return True


user_password = input("Enter password: ")

if validate_password(user_password):
    print("Strong Password")
else:
    print("Weak Password")
```

Result:

```
Weak Password
```

Observation:
The enhanced version significantly improves security by enforcing multiple validation rules. The use of regular expressions increases flexibility and maintainability. The program is modular, readable, and follows PEP 8 standards. Compared to the original version, the AI-enhanced version provides stronger validation, improved structure, and better real-world applicability